

Kubernetes – Core Pod Lifecycle & Storage

#week_one – Mastering Pod Management

duration: 1 week

ForgTech company wants to test your ability to deliver their requirements into a local Kubernetes environment. This will help you build a **good reputation** within the engineering team. The purpose of this task is to prove your capabilities in managing the complete lifecycle of a **Pod**, ensuring high availability through advanced health checks and implementing persistent storage solutions.

Company Scenario

The **ForgTech** Cloud Team is moving toward a "Container-First" strategy for all internal microservices. However, the initial deployments have suffered from intermittent downtime during startup and lack data persistence when pods are restarted. To maintain our **standard of excellence**, you are required to design a pod specification that is resilient, resource-aware, and capable of handling stateful data.

Mission Description

You are tasked with building a robust application environment within a local cluster. You must configure pods that can signal their internal health to the Kubernetes control plane, manage their own resource consumption to avoid cluster-wide starvation, and mount persistent volumes to ensure data survives pod crashes. This is an assignment to build a production-ready pod manifest, not a basic tutorial.

Technical Requirements

1. Initialize a local Kubernetes environment using **kind** or **minikube**. **Do not use cloud-managed services** [Instructions].
2. **Pod Creation:** Define a multi-container pod (Sidecar pattern) utilizing a lightweight image (e.g., `nginx` or `alpine`).
3. **Health Probes:** Implement all three probe types to manage the pod lifecycle:
 - **Startup Probe:** To handle slow-starting legacy components.
 - **Readiness Probe:** To ensure the pod only receives traffic when the application is fully initialized.
 - **Liveness Probe:** To automatically restart the container if the application becomes unresponsive.
4. **Resource Management:** Define explicit **Requests and Limits** for CPU and Memory. The configuration must prevent "noisy neighbor" behavior while allowing for necessary bursts.
5. **Persistent Storage:** Configure a **Volume** and a **VolumeMount**. For this local lab, use a `PersistentVolume` (PV) and `PersistentVolumeClaim` (PVC) backed by local storage to ensure data persistence across pod restarts.
6. **Operational Interaction:** Demonstrate the ability to work with a running pod by executing commands internally (e.g., `kubectl exec`) to verify the mounted volume's integrity.

Constraints & Standards

- **Environment Rules:** Use native Kubernetes YAML manifests.
- **Metadata Standards:** Every resource (Pods, PVCs, PVs) MUST include the following labels:
 - **Key:** "Environment", **Value:** "terraformChamps"
 - **Key:** "Owner", **Value:** "<Your_first_name>"
- **Code Quality:** Manifests must be clean, modular, and follow standard YAML indentation.
- **Documentation:** Build a **personal document** consisting of what you learned with deep details regarding the differences between the three probe types to assist you in refreshing your knowledge later.

Bonus (Optional)

1. **Architecture Diagram:** Build a diagram showing the relationship between the Pod, its Probes, and the Volume lifecycle.
2. **Technical Blog:** Create a post on Hashnode or DEV explaining how **Requests vs. Limits** affect the Kubernetes Scheduler.
3. **ConfigMap Integration:** Inject environment variables into your pod using a `ConfigMap`.

References

- [Kubernetes Documentation – Pods](#)
 - [Configure Liveness, Readiness and Startup Probes](#)
 - [Resource Management for Pods and Containers](#)
 - [Storage Volumes](#)
 - [Stressing a Kubernetes Pod](#)
 - [Noisy Neighbors](#)
-